

Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»

Институт информатики и кибернетики

Кафедра технической кибернетики

Отчет по лабораторной работе №6

Дисциплина: «ООП»

Тема «Разработка приложения, формирующее в одном потоке вычислений
набор заданий для интегрирования, а во втором потоке – вычисляющее
значения интегралов функций»

Выполнил: Куликов Степан
Дмитриевич

Группа: 6201-120303D

Самара, 2025

Задание на лабораторную работу

Задание 1

Добавьте в класс Functions метод, возвращающий значение интеграла функции, вычисленное с помощью численного метода.

```
public static double integrate(Function f, double left, double right, double step) {
    if (left < f.getLeftDomainBorder() || right > f.getRightDomainBorder()) {
        throw new IllegalArgumentException("Интервал интегрирования выходит
за границы функции");
    }
    if (step <= 0) {
        throw new IllegalArgumentException("Шаг дискретизации должен быть
положительным");
    }

    double integral = 0.0;
    double x = left;

    while (x < right) {
        double nextX = Math.min(x + step, right);
        double y1 = f.getFunctionValue(x);
        double y2 = f.getFunctionValue(nextX);

        integral += (y1 + y2) * (nextX - x) / 2.0;
        x = nextX;
    }

    return integral;
}
```

Задание 2

Создайте пакет threads, в котором будут размещены классы, связанные с потоками.

```
package functions.threads;
import functions.Function;

public class Task {
    private Function function;
```

```
private double leftX;
private double rightX;
private double step;
private int count;

public Task() {}

public Function getFunction() {
    return function;
}

public void setFunction(Function function) {
    this.function = function;
}

public double getLeftX() {
    return leftX;
}

public void setLeftX(double leftX) {
    this.leftX = leftX;
}

public double getRightX() {
    return rightX;
}

public void setRightX(double rightX) {
    this.rightX = rightX;
}

public double getStep() {
    return step;
}

public void setStep(double step) {
    this.step = step;
}
```

```

    public int getCount() {
        return count;
    }

    public void setCount(int count) {
        this.count = count;
    }
}

public class Main {

    public static void main(String[] args) {

        ...

        System.out.println("\nПРОВЕРКА ИНТЕГРИРОВАНИЯ");

        double integral = Functions.integrate(exp, 0, 1, 0.1);
        System.out.printf("Интеграл exp(x) на [0,1] с шагом 0.1  $\approx$  %.8f\n", integral);

        double theoretical = Math.exp(1) - Math.exp(0); //  $e^1 - e^0 = e - 1$ 
        double integrateStep = 1.0;
        double diff;
        do {
            integrateStep /= 2.0;
            integral = Functions.integrate(exp, 0, 1, integrateStep);
            diff = Math.abs(integral - theoretical);
        } while (diff >= 1e-7);

        System.out.printf("Для точности 7-го знака после запятой нужен шаг  $\approx$  %.10f,\n",
            integrateStep, integral);
        System.out.println("Запуск последовательной проверки (nonThread)");
        nonThread();
        System.out.println("Проверка завершена.");

    }

    public static void nonThread() {
        Random rnd = new Random();
        Task task = new Task();
    }
}

```

```

task.setCount(100);

for (int i = 0; i < task.getCount(); i++) {
    double base = 1.0 + 9.0 * rnd.nextDouble();
    Function logFunc = new Log(base);
    task.setFunction(logFunc);

    double left = 100.0 * rnd.nextDouble();
    task.setLeftX(left);

    double right = 100.0 + 100.0 * rnd.nextDouble();
    task.setRightX(right);

    double step = rnd.nextDouble();
    if (step == 0) step = 0.01;
    task.setStep(step);

    System.out.printf("Source %.4f %.4f %.4f\n", left, right, step);

    try {
        double result = Functions.integrate(
            task.getFunction(), task.getLeftX(), task.getRightX(), task.getStep()
        );
        System.out.printf("Result %.4f %.4f %.4f %.8f\n", left, right, step, result);
    } catch (IllegalArgumentException e) {
        System.out.println("Ошибка: " + e.getMessage());
    }
}
}

```

Задание 3

В пакете `threads` создайте два следующих класса. При их реализации воспользуйтесь фрагментами последовательной версии программы из предыдущего задания.

```

package functions.threads;

import functions.Function;
import functions.basic.Log;

```

```

import java.util.Random;

public class SimpleGenerator implements Runnable {

    private final Task task;
    private final Random rnd = new Random();

    public SimpleGenerator(Task task) {
        this.task = task;
    }

    public void run() {
        for (int i = 0; i < task.getCount(); i++) {
            synchronized (task) {

                double base = 1.0 + 9.0 * rnd.nextDouble();
                Function logFunc = new Log(base);
                task.setFunction(logFunc);

                double left = 100.0 * rnd.nextDouble();
                task.setLeftX(left);

                double right = 100.0 + 100.0 * rnd.nextDouble();
                task.setRightX(right);

                double step = rnd.nextDouble();
                if (step == 0) step = 0.01;
                task.setStep(step);

                System.out.printf("Source %.4f %.4f %.4f%n", left, right, step);
            }

            try { Thread.sleep(1); } catch (InterruptedException ignored) {}
        }
    }
}

```

```

    }
}

package functions.threads;

import functions.Function;
import functions.Functions;

public class SimpleIntegrator implements Runnable {

    private final Task task;

    public SimpleIntegrator(Task task) {
        this.task = task;
    }

    public void run() {
        for (int i = 0; i < task.getCount(); i++) {
            double left, right, step;
            Function func;

            synchronized (task) {

                func = task.getFunction();
                left = task.getLeftX();
                right = task.getRightX();
                step = task.getStep();
            }

            try {
                double result = Functions.integrate(func, left, right, step);
                System.out.printf("Result %.4f %.4f %.4f %.8f\n", left, right, step,
result);
            } catch (IllegalArgumentException e) {
                System.out.println("Ошибка интегрирования: " + e.getMessage());
            }

            try { Thread.sleep(1); } catch (InterruptedException ignored) {}
        }
    }
}

```

```

    }
}

public class Main {

    public static void main(String[] args) {

        ...

        System.out.println("\nЗапуск поточной версии");
        simpleThreads();

    }

    public static void simpleThreads() {
        Task task = new Task();
        task.setCount(100);

        Thread generatorThread = new Thread(new SimpleGenerator(task));
        Thread integratorThread = new Thread(new SimpleIntegrator(task));

        generatorThread.setPriority(Thread.MAX_PRIORITY);
        integratorThread.setPriority(Thread.MIN_PRIORITY);

        generatorThread.start();
        integratorThread.start();

        try {
            generatorThread.join();
            integratorThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Поточная проверка завершена.");
    }

}

```

Задание 4

Определите причину того, что не все сгенерированные задания оказываются выполнены интегрирующим потоком.

Для устранения этой проблемы потребуется дополнительный объект, представляющий собой одноместный семафор, различающий операции чтения и записи в защищаемый объект (пример класса таких объектов был приведён в лекции).

В пакете `threads` создайте два следующих класса. При их реализации воспользуйтесь фрагментами классов из предыдущего задания.

```
package functions.threads;

import functions.basic.Log;
import functions.Function;
import java.util.Random;

public class Generator extends Thread {

    private final Task task;
    private final SemaphoreSlot semaphore;
    private final Random rnd = new Random();

    public Generator(Task task, SemaphoreSlot semaphore) {
        this.task = task;
        this.semaphore = semaphore;
    }

    public void run() {
        for (int i = 0; i < task.getCount(); i++) {
            try {
                semaphore.acquireWrite();

                // Генерируем задание
                double base = 1.0 + 9.0 * rnd.nextDouble();
                Function logFunc = new Log(base);
                task.setFunction(logFunc);

                double left = 100.0 * rnd.nextDouble();
```

```

        double right = 100.0 + 100.0 * rnd.nextDouble();
        double step = rnd.nextDouble();
        if (step == 0) step = 0.01;

        task.setLeftX(left);
        task.setRightX(right);
        task.setStep(step);

        System.out.printf("Source %.4f %.4f %.4f%n", left, right, step);

        semaphore.releaseWrite();

        Thread.sleep(1);

    } catch (InterruptedException e) {
        System.out.println("Generator прерван");
        break;
    }
}
}
}

package functions.threads;

import functions.Function;
import functions.Functions;

public class Integrator extends Thread {

    private final Task task;
    private final SemaphoreSlot semaphore;

    public Integrator(Task task, SemaphoreSlot semaphore) {
        this.task = task;
        this.semaphore = semaphore;
    }

    public void run() {
        for (int i = 0; i < task.getCount(); i++) {

```

```

try {
    semaphore.acquireRead();

    Function func = task.getFunction();
    double left = task.getLeftX();
    double right = task.getRightX();
    double step = task.getStep();

    double result = Functions.integrate(func, left, right, step);
    System.out.printf("Result %.4f %.4f %.4f %.8f%n", left, right, step,
result);

    semaphore.releaseRead();

    Thread.sleep(1);

} catch (InterruptedException e) {
    System.out.println("Integrator прерван");
    break;
} catch (IllegalArgumentException e) {
    System.out.println("Ошибка интегрирования: " + e.getMessage());
}
}
}
}

package functions.threads;

public class SemaphoreSlot {
    private boolean available = false;

    public synchronized void acquireWrite() throws InterruptedException {
        while (available) {
            wait();
        }
    }

    public synchronized void releaseWrite() {
        available = true;
    }
}

```

```

        notifyAll();
    }

    public synchronized void acquireRead() throws InterruptedException {
        while (!available) {
            wait();
        }
    }

    public synchronized void releaseRead() {
        available = true;
        notifyAll();
    }
}

public class Main {

    public static void main(String[] args) {
        ...
        complicatedThreads();
    }

    public static void complicatedThreads() {
        Task task = new Task();
        task.setCount(100);

        SemaphoreSlot semaphore = new SemaphoreSlot();

        Generator gen = new Generator(task, semaphore);
        Integrator integ = new Integrator(task, semaphore);

        gen.setPriority(Thread.MAX_PRIORITY);
        integ.setPriority(Thread.MIN_PRIORITY);

        gen.start();
        integ.start();
    }
}

```

```

try {
    Thread.sleep(50);

    gen.interrupt();
    integ.interrupt();

    gen.join();
    integ.join();
} catch (InterruptedException e) {
    e.printStackTrace();
}

System.out.println("Поточная проверка с семафором завершена.");
}
}

```

Команды в консоли:

C:\Users\fael7\Desktop\лр6\Lab-6-2025>javac Main.java

C:\Users\fael7\Desktop\лр6\Lab-6-2025>java Main

...

ПРОВЕРКА ИНТЕГРИРОВАНИЯ

Интеграл $\exp(x)$ на $[0,1]$ с шагом 0.1 ? 1,71971349

Для точности 7-го знака после запятой нужен шаг ? 0,0004882813, интеграл ? 1,7182818626

Запуск последовательной проверки (nonThread)

Source 57,3305 131,8961 0,6779

Result 57,3305 131,8961 0,6779 161,64361143

...

Source 73,2177 117,6502 0,1485

Result 73,2177 117,6502 0,1485 114,49421281

Проверка завершена.

Запуск поточной версии

Source 8,9520 115,9545 0,9091

Result 8,9520 115,9545 0,9091 233,57542053

...

Result 57,2096 113,5968 0,5484 121,67742969

Result 57,2096 113,5968 0,5484 121,67742969

Поточная проверка завершена.

Source 94,4103 100,5492 0,8159

Result 94,4103 100,5492 0,8159 17,63148520

...

Result 12,7673 184,2336 0,5922 632,01047291

Source 27,4712 174,0563 0,1094

Integrator прерван

Generator прерван

Поточная проверка с семафором завершена.

Работа программы завершена.